Prerequisites

Install Work IQ

Prepare to develop an app in Visual Studio Code

Explore Workplace Intelligence Scenarios

Explore and Experiment

Understanding the Code

Clean Up

Troubleshooting

Work IQ - Workplace intelligence for AI agents

In this lab, you'll build an AI agent that accesses your Microsoft 365 workplace data using **Work IQ** - Microsoft's contextual intelligence layer built on the Model Context Protocol (MCP). You'll create a workplace intelligence agent that can prepare for meetings, track projects, extract action items, and answer workplace questions using real M365 data.

This lab takes approximately **40** minutes.

> **Note:** This is an **optional/advanced lab** that requires a Microsoft 365 Copilot license. It's designed for enterprise learners, Microsoft employees, or those with M365 Copilot access. Standard M365 accounts without Copilot will not work.

Prerequisites

Before starting this lab, ensure you have:

- Basic understanding of AI agents and the Model Context Protocol (MCP)
- **Microsoft 365 with Copilot License**
- IT admin approval for Work IQ (organizational accounts only)
- Node.js 18 or later installed
- Python 3.13 installed
- Azure CLI installed (authenticated with `az login`)
- Active M365 data (emails, meetings, Teams chats) to query

> * Python 3.14 isn't supported yet: some dependencies have no 3.14 build. This lab was tested with Python 3.13.12.

> **Important:** Work IQ **only works** with Microsoft 365 Copilot-enabled accounts. You cannot complete this lab without Copilot.

Install Work IQ

1. Open your terminal or command prompt.

2. Install Work IQ globally via npm:

Code Copy

```
npm install -g @microsoft/workiq
```

3. Accept the End User License Agreement:

Code Copy

```
workiq accept-eula
```

4. Test your Work IQ installation:

Code Copy

```
workiq ask -q "What meetings do I have today?"
```

5. **If the test succeeds** - You'll see meeting information from your M365 calendar. Continue to the next task!

6. **If you see "Admin consent required":**
    - The command will display a consent URL
    - Send this URL to your IT administrator with the message: "I need Work IQ access for the Microsoft Learn AI Agents lab"
    - Wait for admin approval, then retry the test command
7. **If you see "No M365 Copilot license":**
    - Unfortunately, you cannot complete this lab without a Copilot license
    - You can still read through the instructions to understand the concepts
    - Consider this lab optional and return to it when you have Copilot access

Prepare to develop an app in Visual Studio Code

Now let's use Visual Studio Code to develop an app. The code files for your app have been provided in a GitHub repo.

1. Start Visual Studio Code, and open a terminal window.
2. Enter the command to clone the repo to a local folder (it doesn't matter which folder):

```
git clone https://github.com/MicrosoftLearning/mslearn-ai-agents.git
```

3. When the repository has been cloned, open the folder in Visual Studio Code.

> **Note**: If Visual Studio Code shows you a pop-up message prompting you to trust the code you are opening, select **Yes, I trust the authors** to continue.

4. Wait while additional files are installed to support the Python code projects in the repo (if prompted).

> **Note**: If you are prompted to install required assets to build and debug, select **Not Now**.

5. In the **Explorer** pane, expand the **Labfiles/05b-work-iq-integration/Python** folder.

   The provided files include application code, configuration settings, and the agent client starter code.

6. In the terminal, enter the command to create a Python virtual environment:

```
python -m venv venv
```

7. Activate the virtual environment:

   Windows:

```
venv\Scripts\activate
```

   macOS/Linux:

```
source venv/bin/activate
```

8. Install required Python packages:

```
pip install -r requirements.txt
```

9. Configure your `.env` file:

   In the lab folder, open the `.env` file and update it with your Foundry project endpoint:

```
PROJECT_ENDPOINT=https://your-project.services.ai.azure.com/api/projects/your-id
MODEL_DEPLOYMENT_NAME=gpt-5
```

> **Tip:** To get your endpoint: In VS Code, open the **Foundry Toolkit** extension, right-click on your active project, and select **Copy Endpoint**. The Foundry Toolkit is included in the Foundry Toolkit for VS Code extension.

Verify setup

Ensure you have:

- Work IQ installed and accessible (`workiq --version` works)
- Admin consent approved (or personal M365 account with Copilot)
- `workiq_lab.py` - Main interactive application
- `requirements.txt` - Python dependencies installed
- `.env` file configured with your project endpoint

Explore Workplace Intelligence Scenarios

In this exercise, you'll run a unified interactive application that demonstrates five workplace intelligence scenarios using a single AI agent with Work IQ tools.

Launch the lab application

1. Ensure you're in the lab directory with your virtual environment activated.

2. Run the lab application:

```
python workiq_lab.py
```

3. The application will:

   - Validate Work IQ setup
   - Connect to your Microsoft Foundry project
   - Initialize the Work IQ MCP client
   - Create a workplace intelligence agent
   - Display an interactive menu with 5 scenarios

Meeting Prep scenario

This scenario helps you prepare for meetings by gathering relevant context.

1. From the main menu, select **1 - Meeting Prep**.
2. When prompted, enter a meeting topic or time, such as:

- "my 2pm meeting"
- "Q4 Planning session"
- "team standup"

3. The agent will:

- Find your meeting details (time, attendees, agenda)
- Search recent emails about the topic
- Look for previous meetings on this subject
- Summarize key points and decisions
- Suggest discussion points

4. Review the output and note:

- How sources are cited (emails, meetings, dates)
- How the agent synthesizes information from multiple sources
- The time saved compared to manual searching

Reflection: How does this differ from manually searching your email and calendar?

Project Status scenario

This scenario tracks project updates across your workplace tools.

1. From the main menu, select **2 - Project Status**.
2. Enter a project name you're working on, such as:

- "Website redesign"
- "Q1 OKRs"
- "Customer onboarding"

3. The agent will:

- Search emails and Teams messages about the project
- Find related meetings and their outcomes
- Identify recent decisions and changes
- List blockers or issues mentioned
- Summarize next steps and deadlines

4. Analyze the results:

- How comprehensive is the status update?
- What sources did the agent use?
- Could this be built with traditional APIs? What's the development effort difference?

Action Items scenario

This scenario extracts your open tasks from various sources.

1. From the main menu, select **3 - Action Items**.
2. Choose a time range (or press Enter for "this week"):

- "today"
- "last 3 days"
- "this month"

3. The agent will:

- Search meeting notes for assigned action items
- Look for task-related emails sent to you
- Check Teams messages where you were mentioned
- Identify items with deadlines
- Prioritize by urgency if possible

4. Examine the output:

- Are all your action items captured?
- How accurate is the prioritization?
- Where were action items found (meetings, emails, Teams)?

Combined Intelligence scenario

This scenario demonstrates using **both** Work IQ (workplace data) and Foundry IQ (knowledge base) together.

> **Note:** This scenario requires Azure AI Search configured in your Foundry project with an indexed knowledge base.

1. From the main menu, select **4 - Combined Intelligence**.
2. Enter a topic that exists in both your workplace discussions and official documentation:
    - “remote work policy”
    - “expense reporting”
    - “security guidelines”
3. The agent will:
    - Search workplace data (Work IQ): emails, meetings, Teams discussions
    - Search knowledge base (Foundry IQ): official docs, policies, procedures
    - Compare workplace discussions with official documentation
    - Identify gaps or inconsistencies
    - Provide a comprehensive summary with labeled sources
4. Compare the two perspectives:
    - What’s documented officially vs. discussed informally?
    - Are there any contradictions?
    - Which source is more up-to-date?

Key Insight:

- **Work IQ** tells you what people are actually doing and saying
- **Foundry IQ** tells you what’s officially documented
- **Together** they provide complete context for decision-making

Custom Query scenario

This scenario lets you explore your workplace data with your own questions.

1. From the main menu, select **5 - Custom Query**.
2. Try different types of workplace questions:

    Email searches:

    Code Copy

    ```
    Find emails about the budget from my manager
    ```

    Meeting summaries:

    Code Copy

    ```
    What was decided in yesterday's standup?
    ```

    Team activity:

    Code Copy

    ```
    What did the engineering team discuss this week?
    ```

    Document discovery:

    Code Copy

```
Show me shared documents about security policies
```

3. Experiment with:
    - Different time ranges
    - Different data sources (emails vs. meetings vs. Teams)
    - Different levels of specificity
    - Follow-up questions to refine results
4. Note what works well:
    - Specific queries usually work better than vague ones
    - Including time ranges improves relevance
    - Names and keywords help narrow results

Explore and Experiment

Now that you've completed all scenarios, take 5-10 minutes to explore on your own.

Test edge cases

1. Try queries about data you don't have - how does the agent respond?
2. Ask ambiguous questions - how does the agent handle them?
3. Search for very old information - what are the limits?

Explore different query styles

1. **Very specific**: "Find the email from John about Q3 budget sent on January 15th"
2. **Very broad**: "Tell me about recent developments"
3. **Comparative**: "Compare this week's discussions to last week's"

View Work IQ capabilities

From the main menu, select **6 - View Work IQ Capabilities** to review:

- Architecture overview
- Data sources available
- Security and privacy model
- Work IQ vs. Foundry IQ comparison
- Common use cases

Understanding the Code

Let's examine the key patterns used in this lab.

Pattern 1: Work IQ MCP Client Initialization

Code Copy

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

# Store server parameters for reuse
self.workiq_server_params = StdioServerParameters(
    command="npx",
    args=["-y", "@microsoft/workiq", "mcp"]
)

# Fetch available tools from Work IQ MCP server
async def _fetch():
    async with stdio_client(self.workiq_server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            return tools_result.tools

raw_tools = asyncio.run(_fetch())
```

Rather than maintaining a persistent connection, a new MCP session is opened per operation. `StdioServerParameters` stores the command and arguments used to launch the Work IQ MCP server subprocess each time.

Pattern 2: Creating Agent with Work IQ Tools

Code Copy

```
from azure.ai.projects.models import PromptAgentDefinition, FunctionTool

# Convert MCP tools to FunctionTool objects
workiq_tools = [
    FunctionTool(
        name=tool.name,
        description=tool.description,
        parameters=tool.inputSchema,
    )
    for tool in raw_tools
]

# Create agent with Work IQ tools
self.agent = self.project_client.agents.create_version(
    agent_name="workplace-intelligence-agent",
    definition=PromptAgentDefinition(
        model=self.model_deployment,
        instructions="You are a workplace intelligence assistant...",
        tools=workiq_tools  # Work IQ tools added here
    )
)

# Keep a map of raw tools for lookup during execution
self.raw_tools_map = {tool.name: tool for tool in raw_tools}
```

Each MCP tool is wrapped in a `FunctionTool` object and passed to a `PromptAgentDefinition`. The raw tools map enables efficient lookup when the agent calls a tool by name.

Pattern 3: Executing Queries with Responses API

Code Copy

```
# Create conversation
conversation = self.openai_client.conversations.create(
    items=[{"type": "message", "role": "user", "content": query}]
)

# Create response with agent
response = self.openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference": {"name": self.agent.name, "type": "agent_reference"}}
)
```

This uses the Responses API pattern (not the old Runs/Threads pattern) for cleaner agent execution.

Pattern 4: Tool Call Loop

After the initial response, the agent may request one or more Work IQ tool calls. These must be executed and fed back to continue the conversation:

Code Copy

```
from openai.types.responses.response_input_param import FunctionCallOutput

while True:
    if response.status == "failed":
        break

    input_list = []
    for item in response.output:
        if item.type == "function_call":
            kwargs = json.loads(item.arguments)

            # Call the Work IQ tool via MCP
            async def _execute():
                async with stdio_client(self.workiq_server_params) as (read, write):
                    async with ClientSession(read, write) as session:
                        await session.initialize()
                        return await session.call_tool(item.name, kwargs)

            result = asyncio.run(_execute())
            input_list.append(
                FunctionCallOutput(
                    type="function_call_output",
                    call_id=item.call_id,
                    output=result.content[0].text,
                )
            )

    if input_list:
        # Send tool results back and continue
        response = self.openai_client.responses.create(
            input=input_list,
            previous_response_id=response.id,
            extra_body={"agent_reference": {"name": self.agent.name, "type": "agent_reference"}}
        )
    else:
        break  # No more tool calls - final response ready
```

The loop continues until the agent produces a response with no pending function calls, at which point `response.output_text` contains the final answer.

Clean Up

The lab automatically cleans up the agent when you exit:

Code Copy

```
self.openai_client.agents.delete_version(
    agent_name=self.agent.name,
    version=self.agent.version
)
```

No Azure resources are created in this lab (Work IQ uses your M365 license), so no additional cleanup is needed.

Troubleshooting

“Work IQ command not found”

Solution: Install Work IQ:

Code Copy

```
npm install -g @microsoft/workiq
```

“Admin consent required”

Solution:

1. Run `workiq mcp` to get the consent URL
2. Send to your IT admin for approval
3. Or use a personal M365 account with Copilot

“No M365 Copilot license”

Solution: This lab requires Copilot. Either:

- Purchase M365 Copilot license ($30/month)
- Use organizational account with Copilot
- Read through the lab to understand concepts without hands-on

“MCP server not responding”

Solution: Test Work IQ directly:

Code Copy

```
workiq ask -q "What meetings do I have?"
```

If this fails, reinstall:

Code Copy

```
npm install -g @microsoft/workiq
```

“No data returned”

Solution:

- Ensure your M365 account has emails, meetings, Teams activity
- Try broader queries
- Check if your query matches your actual data